

Estructuras de Datos Avanzadas

Banco de 5 ejercicios de casos de uso

Estilo examen practico - dificultad creciente

Instrucciones generales

- La duracion recomendada para cada ejercicio es de 60 a 90 minutos, segun la dificultad indicada.
- La puntuacion de cada apartado se indica junto al enunciado. Cada ejercicio suma 10 puntos.
- No se incluyen soluciones. La interpretacion del enunciado forma parte del entrenamiento.
- No se pueden anadir nuevas clases publicas. Solo se pueden anadir propiedades y metodos privados en las clases ya dadas.
- No se pueden modificar las cabeceras de los metodos ya dados.
- Se valorara especialmente la eleccion eficiente de estructuras de datos: HashMap, HashSet, TreeMap, TreeSet, PriorityQueue y grafos manuales con mapas.
- Todas las excepciones indicadas en un ejercicio deberan lanzar el mensaje exacto: "Not valid!".
- Los metodos que devuelvan Iterable o Collection podran devolver una vista o una copia, salvo que el enunciado diga lo contrario.

Estructura del documento

Operacion	Descripcion	Puntos
Ejercicio 1 RegistroCNPPlus	Registro con busquedas por DNI, rangos de nota y ranking. Dificultad baja-media.	10
Ejercicio 2 InteractionDetectorPlus	Contactos con fechas, propagacion por BFS y arbol de interes. Dificultad media.	10
Ejercicio 3 SafeBroadcastNet	Routers, mensajes de difusion, TTL, borrado por fecha y nodo central. Dificultad media-alta.	10
Ejercicio 4 URJCFlightsPlus	Grafo dirigido de vuelos, indices temporales y consultas por escalas. Dificultad alta.	10
Ejercicio 5 Emergencias112	Grafo de bases, incidentes por prioridad/fecha, reasignacion y centro operativo. Dificultad muy alta.	10

Ejercicio 1 [10 puntos]

Caso de uso: oposiciones al Cuerpo Nacional de Policia - RegistroCNPPlus

El Cuerpo Nacional de Policia necesita ampliar su sistema de registro de opositores. Cada opositor tiene un DNI unico, un nombre, una nota de teoria, una nota fisica y una nota psicotecnica. La nota media se calcula como la media aritmetica de esas tres calificaciones. Se desea implementar la clase **RegistroCNPPlus**, que almacena opositores y permite consultar rapidamente por DNI y por rangos de nota media.

El registro debe evitar opositores repetidos. Dos opositores se consideran repetidos si tienen el mismo DNI. Ademias, las consultas por nota deben poder resolverse de forma eficiente, ya que se presentan muchos opositores.

Operaciones solicitadas

Operacion	Descripcion	Puntos
<code>public RegistroCNPPlus(Iterable<Opositor> opositores)</code>	Inicializa el registro con los opositores recibidos. Si el iterable es null, se lanzara <code>IllegalArgumentException</code> .	0,75
<code>public Iterable<Opositor> opositores()</code>	Devuelve un iterable con todos los opositores almacenados.	0,50
<code>public int size()</code>	Devuelve el numero de opositores en el registro.	0,50
<code>public Opositor getOpositor(Integer dni)</code>	Devuelve el opositor con el DNI indicado, o null si no existe. Si el DNI es null, se lanzara <code>IllegalArgumentException</code> .	0,75
<code>public boolean addOpositor(Opositor opositor)</code>	Añade un opositor al registro. Si ya existe otro opositor con el mismo DNI, devuelve false. Si se añade correctamente, devuelve true.	1,00
<code>public boolean removeOpositor(Integer dni)</code>	Elimina el opositor con el DNI indicado. Devuelve true si existia y false en caso contrario.	1,00
<code>public double getMinNotaMedia()</code>	Devuelve la nota media minima del registro. Si el registro esta vacio, se lanzara <code>IllegalStateException</code> .	0,75
<code>public double getMaxNotaMedia()</code>	Devuelve la nota media maxima del registro. Si el registro esta vacio, se lanzara <code>IllegalStateException</code> .	0,75
<code>public Iterable<Opositor> getOpositoresConNotaMediaSuperiorA(double notaMedia)</code>	Devuelve los opositores con nota media estrictamente superior a la indicada.	1,00
<code>public Iterable<Opositor> getOpositoresConNotaMediaEnElRango(double min, double max)</code>	Devuelve los opositores con nota media mayor o igual que min y menor o igual que max. Si min > max, se lanzara <code>IllegalArgumentException</code> .	1,25
<code>public Iterable<Opositor> getTopK(int k)</code>	Devuelve como maximo los k mejores opositores, ordenados por nota media descendente y, en caso de empate, por DNI ascendente.	1,75

Consideraciones adicionales

- La clase `RegistroCNPPlus` podra tener como maximo dos propiedades privadas.
- Se recomienda mantener un indice por DNI y otro indice ordenado por nota media.
- Si cambia la posicion de un opositor en una estructura ordenada, se debe eliminar y reinsertar para no romper el orden.
- Todas las excepciones de este ejercicio deben lanzar el mensaje: "Not valid!".

Ejercicio 2 [10 puntos]

Caso de uso: detector de interacciones del CNI - InteractionDetectorPlus

El CNI quiere construir una aplicación que registre acercamientos entre personas y permita detectar grupos de interés. Cada acercamiento tiene una fecha, una duración en minutos y dos identificadores de persona. Una persona se considera sospechosa si existe una cadena de contactos de interés que la conecta con una persona raíz.

Un contacto es de interés si se produjo en una fecha igual o posterior a una fecha F y si la duración acumulada entre las dos mismas personas en esa fecha es mayor o igual que M minutos. La propagación se realiza como un recorrido en anchura o profundidad sobre el grafo no dirigido de personas.

Figura 1. Red de contactos de ejemplo

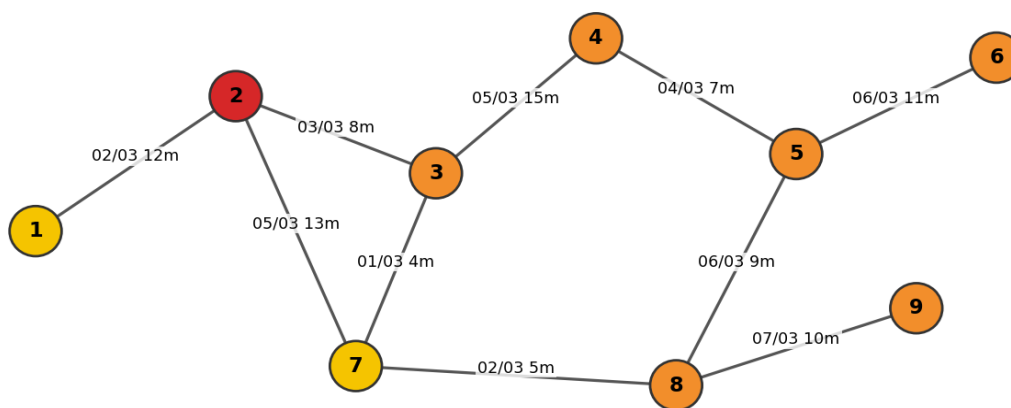


Figura 1. Red de contactos de ejemplo. El nodo rojo es la persona raíz y los nodos naranjas serían alcanzables para determinados umbrales.

En la clase Interactions se pide:

Operación	Descripción	Puntos
[0,50 puntos] Propiedades privadas	Definir las estructuras de datos adecuadas para almacenar, por fecha, los minutos acumulados entre dos personas.	0,50
public void addInteraction(LocalDate date, int minutes)	Añade una interacción. Si ya existe una interacción en la misma fecha, se suman los minutos a los ya almacenados. Fechas null o minutos negativos producen excepción.	1,00
public boolean isOfInterest(LocalDate date, int minutes)	Devuelve true si existe al menos una fecha igual o posterior a date con minutos acumulados mayores o iguales que minutes.	1,25
public int totalMinutesFrom(LocalDate date)	Devuelve la suma de minutos de todas las interacciones almacenadas con fecha igual o posterior a date.	1,25

En la clase InteractionDetectorPlus se pide:

Operación	Descripción	Puntos
[0,50 puntos] Propiedades privadas	Seleccionar las estructuras de datos adecuadas para guardar las personas y las interacciones entre pares de personas.	0,50
public void storeContact(LocalDate date, int minutes, int idPersonA, int idPersonB)	Almacena un contacto entre dos personas. Si las personas no existían previamente, se incorporan al sistema. No se admiten contactos de una persona consigo misma.	1,00
public Collection<Integer> getInterestTree(int idRoot, LocalDate date, int minutes)	Devuelve todos los identificadores que pertenecen al árbol de interés de idRoot. La raíz siempre debe aparecer si existe en el sistema. Si la persona raíz no existe, devuelve una colección vacía.	2,00

Operacion	Descripcion	Puntos
<pre>public Map<Integer, Integer> getInterestLevels(int idRoot, LocalDate date, int minutes)</pre>	Devuelve un mapa que asocia a cada persona sospechosa su distancia minima en numero de contactos desde idRoot.	1,50
<pre>public Integer mostConnectedSuspect(int idRoot, LocalDate date, int minutes)</pre>	Devuelve, entre las personas del arbol de interes, aquella que tiene mas contactos de interes con otras personas del mismo arbol. En caso de empate, devuelve el id menor. Si el arbol esta vacio, devuelve null.	1,00

Consideraciones adicionales

- El grafo debe considerarse no dirigido: si A ha tenido contacto con B, B tambien esta conectado con A.
- Se valorara el uso de `HashMap<Integer, HashMap<Integer, Interactions>>` o estructuras equivalentes.
- No se pueden modificar las interfaces suministradas. Se pueden anadir metodos privados auxiliares.
- Todas las excepciones de este ejercicio deben lanzar el mensaje: "Not valid!".

Ejercicio 3 [10 puntos]

Caso de uso: URJCNetServices - SafeBroadcastNet

URJCNetServices administra una red de routers conectados mediante enlaces no dirigidos. Los routers intercambian mensajes de difusion que no deben circular indefinidamente. Cada router almacena los mensajes que ya ha visto y permite borrar mensajes antiguos. Además, la red debe permitir decidir que router es mejor para iniciar una difusion.

Cada mensaje contiene un identificador unico, una fecha de envio, la IP del router emisor y un texto. Dos mensajes con el mismo identificador se consideran el mismo mensaje, aunque el resto de campos difiera.

Figura 2. Red de routers de ejemplo

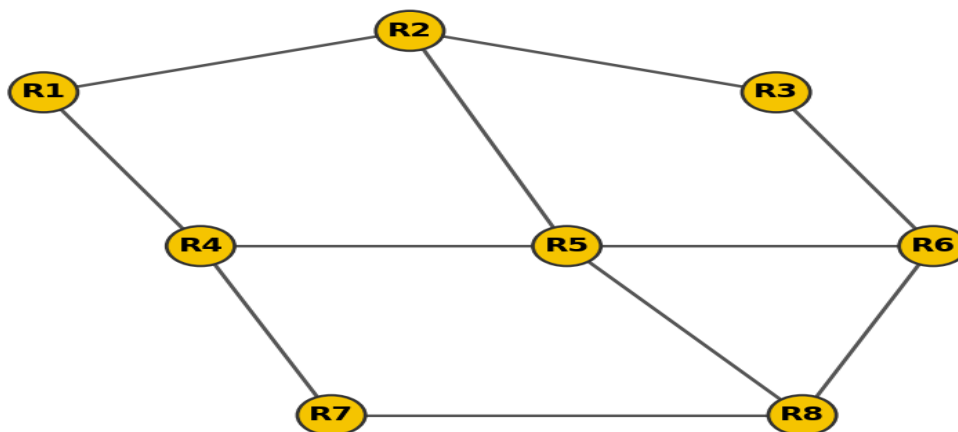


Figura 2. Red de routers de ejemplo. Todas las conexiones tienen coste 1 salvo que el proyecto indique otra cosa.

En la clase DateComparator se pide:

Operacion	Descripcion	Puntos
<pre>public int compare(LocalDateTime d1, LocalDateTime d2)</pre>	Compara dos fechas en orden ascendente. El metodo debe declarar una variable local llamada quienVaAntes. Fechas null no son validas.	1,00

En la clase Router se pide:

Operacion	Descripcion	Puntos
[0,25 puntos] Propiedades privadas	Seleccionar las estructuras necesarias para recordar mensajes por identificador y por fecha de envio.	0,25
<pre>public boolean mensajeDeDifusion(BroadcastMessage message)</pre>	Si el mensaje no se habia recibido antes, lo almacena y devuelve true. Si ya estaba en memoria, devuelve false. Mensaje null no es valido.	1,00
<pre>public int borrarMensajesAntiguos(LocalDateTime date)</pre>	Elimina todos los mensajes con fecha igual o anterior a date. Devuelve cuantos mensajes se han borrado.	1,75
<pre>public Iterable<BroadcastMessage> mensajesEntre(LocalDateTime from, LocalDateTime to)</pre>	Devuelve los mensajes recibidos con fecha entre from y to, ambos incluidos, ordenados por fecha ascendente.	1,00
<pre>public boolean conoceMensaje(String messageId)</pre>	Devuelve true si el router ya ha recibido un mensaje con ese identificador. Identificadores null no son validos.	0,50

En la clase SafeBroadcastNet se pide:

Operacion	Descripcion	Puntos
[0,25 puntos] Propiedades privadas	Seleccionar las estructuras para representar la red y acceder a los routers por IP.	0,25

Operacion	Descripcion	Puntos
<code>public boolean addRouter(Router router)</code>	Anade un router a la red. Si ya existe un router con la misma IP, devuelve false.	0,75
<code>public boolean addConnection(String ipA, String ipB)</code>	Anade una conexion no dirigida entre dos routers existentes. Si alguno no existe, devuelve false. Si la conexion ya existia, devuelve false.	1,00
<code>public Collection<String> routersAlcanzablesConTTL(String ipOrigen, int ttl)</code>	Devuelve las IP de los routers alcanzables desde ipOrigen usando como maximo ttl saltos. El router origen debe aparecer con distancia 0.	1,50
<code>public String centralNode()</code>	Devuelve la IP del nodo central: el router que minimiza la distancia maxima al resto. Si la red esta vacia, se lanza excepcion. Si no es conexas, devuelve null. En empate, devuelve la IP lexicograficamente menor.	2,00
<code>public Map<String, String> broadcastTree(String ipOrigen)</code>	Devuelve un mapa hijo-padre que representa un arbol de difusion obtenido por BFS desde ipOrigen. La raiz aparecera asociada a null. Si la red no es conexas, solo se incluyen los routers alcanzables.	1,00

Consideraciones adicionales

- Se puede implementar el grafo manualmente con `HashMap<String, HashSet<String>>` o usando el grafo proporcionado por el entorno.
- Si se usa `AdjacencyMapGraph`, las aristas deben contener objetos `Double` con valor 1.0.
- Los recorridos por la red deben evitar ciclos mediante un conjunto de visitados.
- Todas las excepciones de este ejercicio deben lanzar el mensaje: "Not valid!".

Ejercicio 4 [10 puntos]

Caso de uso: URJCFlightsPlus - rutas, escalas e indice temporal

URJCFlights gestiona aeropuertos y vuelos directos. Los vuelos son dirigidos: un vuelo de A a B no implica que exista un vuelo de B a A. Cada vuelo tiene un codigo unico, aeropuerto origen, aeropuerto destino, fecha y hora de salida, precio y duracion.

La compania quiere consultar destinos alcanzables con un numero maximo de escalas, borrar vuelos antiguos y buscar vuelos disponibles en intervalos de fechas. Se desea implementar la clase **URJCFlightsPlus**.

Figura 3. Rutas dirigidas de ejemplo

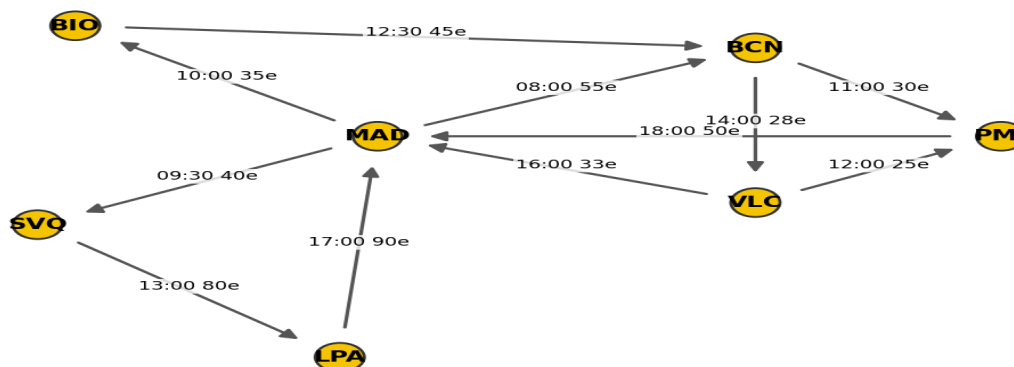


Figura 3. Rutas dirigidas de ejemplo. Las etiquetas muestran hora y precio orientativos.

Operacion	Descripcion	Puntos
[0,50 puntos] Propiedades privadas	Seleccionar las estructuras adecuadas para acceder por codigo de vuelo, por aeropuerto y por fecha de salida.	0,50
public boolean addAirport(String airportCode)	Anade un aeropuerto. Si ya existe, devuelve false. Codigos null o vacios no son validos.	0,75
public boolean addFlight(Flight flight)	Anade un vuelo dirigido. El codigo de vuelo debe ser unico y los aeropuertos origen y destino deben existir. Si no se puede anadir, devuelve false.	1,25
public boolean removeFlight(String flightCode)	Elimina un vuelo por codigo y actualiza todos los indices. Devuelve true si existia.	1,00
public int removeFlightsBefore(LocalDateTime date)	Elimina todos los vuelos con salida estrictamente anterior a date y devuelve cuantos se han borrado.	1,25
public Iterable<Flight> flightsBetween(LocalDateTime from, LocalDateTime to)	Devuelve los vuelos con salida entre from y to, ambos incluidos, ordenados por fecha ascendente.	1,25
public Collection<String> reachableAirports(String origin, int maxStops)	Devuelve los aeropuertos alcanzables desde origen usando como maximo maxStops escalas. Debe usarse BFS sobre el grafo dirigido.	1,25
public boolean existsRouteAfter(String origin, String destination, LocalDateTime date, int maxStops)	Devuelve true si existe una ruta desde origen hasta destination con como maximo maxStops escalas y usando solo vuelos con salida igual o posterior a date.	1,50
public Flight cheapestDirectFlight(String origin, String destination, LocalDateTime from, LocalDateTime to)	Devuelve el vuelo directo mas barato entre origen y destination dentro del intervalo indicado. En empate, devuelve el de salida mas temprana. Si no existe, devuelve null.	1,25

Consideraciones adicionales

- El grafo es dirigido y puede haber varios vuelos entre los mismos aeropuertos.
- Para el indice temporal se recomienda `TreeMap<LocalDateTime, Set<Flight>>` o estructura equivalente.
- Todos los indices deben permanecer coherentes tras borrar vuelos.
- Todas las excepciones de este ejercicio deben lanzar el mensaje: "Not valid!".

Ejercicio 5 [10 puntos]

Caso de uso: Emergencias112 - bases, incidentes y rutas de atencion

El servicio Emergencias112 coordina varias bases distribuidas por una region. Las bases estan conectadas mediante carreteras. Cada base tiene un identificador, un numero de ambulancias disponibles y una zona asignada. Ademias, el sistema recibe incidentes con fecha, localidad, gravedad y estado.

La aplicacion debe elegir rapidamente el siguiente incidente a atender, buscar bases cercanas por carretera y mantener un indice temporal de incidentes. Este ejercicio mezcla mapas, grafos, colas de prioridad e indices ordenados.

Figura 4. Bases y carreteras de ejemplo

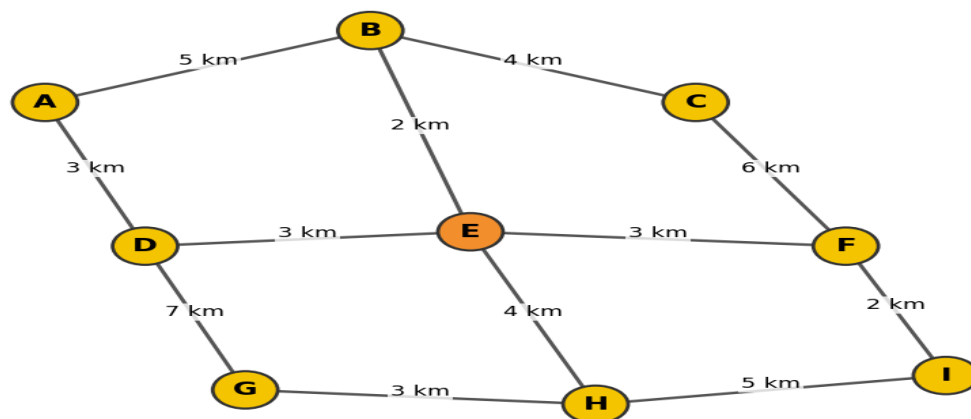


Figura 4. Bases y carreteras de ejemplo. La base E aparece destacada como posible centro operativo.

En la clase IncidentPriorityComparator se pide:

Operacion	Descripcion	Puntos
<pre>public int compare(Incident i1, Incident i2)</pre>	Ordena incidentes por gravedad descendente, fecha ascendente y, en empate, identificador ascendente. Incidentes null no son validos.	1,00

En la clase Emergencias112 se pide:

Operacion	Descripcion	Puntos
[0,50 puntos] Propiedades privadas	Seleccionar estructuras para bases por id, grafo de carreteras, incidentes por id, incidentes por fecha y cola de prioridad de pendientes.	0,50
<pre>public boolean addBase(Base base)</pre>	Anade una base. Si ya existe una base con el mismo id, devuelve false.	0,75
<pre>public boolean addRoad(String baseA, String baseB)</pre>	Anade una carretera no dirigida entre dos bases existentes. Si alguna base no existe o la carretera ya existe, devuelve false.	0,75
<pre>public boolean registerIncident(Incident incident)</pre>	Registra un incidente pendiente. El identificador debe ser unico. Deben actualizarse el indice por id, el indice por fecha y la cola de prioridad.	1,25
<pre>public Incident attendNextIncident()</pre>	Devuelve y marca como atendido el incidente pendiente con mayor prioridad. Si no hay incidentes pendientes, devuelve null.	1,25
<pre>public Collection<Incident> incidentsBetween(LocalDateTime from, LocalDateTime to)</pre>	Devuelve los incidentes registrados entre from y to, ambos incluidos, ordenados por fecha ascendente.	1,00
<pre>public Collection<String> nearbyBases(String baseId, int maxRoads)</pre>	Devuelve las bases alcanzables desde baseId usando como maximo maxRoads carreteras. Debe incluir la base de origen.	1,00

Operacion	Descripcion	Puntos
<code>public String assignNearestAvailableBase(String incidentId)</code>	Asigna al incidente pendiente la base disponible mas cercana a su base de referencia. Se usa BFS. En empate por distancia, gana la base con id lexicograficamente menor. Si se asigna, reduce en uno las ambulancias disponibles.	1,50
<code>public String operationalCenter()</code>	Devuelve la base que minimiza la distancia maxima al resto de bases. Si el grafo esta vacio, lanza excepcion. Si no es conexo, devuelve null. En empate, gana la base con mas ambulancias disponibles y despues el id menor.	1,25
<code>public boolean cancelIncident(String incidentId)</code>	Cancela un incidente pendiente y actualiza los indices. Si ya estaba atendido o no existe, devuelve false.	0,75

Consideraciones adicionales

- PriorityQueue no reordena automaticamente objetos modificados; si cambia la prioridad de un incidente pendiente, se debe quitar y volver a insertar.
- El borrado o cancelacion de incidentes debe mantener coherentes todos los indices.
- El grafo puede implementarse con `HashMap<String, HashSet<String>>`.
- Se valorara que las consultas de rango temporal se resuelvan mediante `TreeMap`.
- Todas las excepciones de este ejercicio deben lanzar el mensaje: "Not valid!".

Checklist de entrenamiento antes de entregar

- He elegido primero la estructura principal: mapa por identificador, grafo, índice ordenado o cola de prioridad.
- He comprobado null, estructuras vacías, duplicados y rangos inválidos.
- He actualizado todos los índices al insertar, borrar o cancelar.
- He usado visitados en cada BFS/DFS para evitar ciclos.
- He controlado los empates que indica cada enunciado.
- Puedo explicar la complejidad de cada método importante.